Final Project - Final Presentation Slide

Understanding and Improving GPT-2 Implementations in NanoGPT

Wang Zeyu

National Taiwan University

Overview

Task 1:

Task 1 Quick Recap

Task 1 RECAP: Multi-head Attention: Slides vs NanoGPT

Lecture Slides Approach:

- Separate projections: $Q = \tilde{Z}W_Q$, $K = \tilde{Z}W_K$, $V = \tilde{Z}W_V$
- Single sequence: $\tilde{Z} \in \mathbb{R}^{T \times d}$
- Per-head attention: $\text{head}_i = \text{Softmax}\left(\frac{Q^{(i)}(K^{(i)})^\top}{\sqrt{d_h}}\right) V^{(i)}$

NanoGPT Implementation:

- **Combined projection**: $[Q|K|V] = xW_{\text{combined}} + b_{\text{combined}}$
  - Single matrix multiplication instead of three
  - Batched computation: $x \in \mathbb{R}^{B \times T \times d}$ (includes batch dimension B)
- **Head parallelization via reshape + transpose**:
  - View: $(B, T, d) \to (B, T, h, d_h)$, then Transpose: $(B, T, h, d_h) \to (B, h, T, d_h)$
  - Treats h heads as extra batch dimension: $B \cdot h$ independent batches
  - All heads computed in parallel (single GPU kernel), no Python loops

Key Differences Summary

Aspect	**Slides**	**NanoGPT**
Q/K/V Projection	3 separate	1 combined
Input Dimension	(T, d)	(B, T, d)
Head Computation	Sequential	Parallel
Attention Method	Standard	Flash Attention option

Why these differences?

- **GPU efficiency**: Batched matrix operations utilise GPU parallelism
- **Memory optimisation**: Combined projections reduce memory overhead
- **Speed**: Parallel head computation eliminates sequential loops

Result: Same mathematical operation, significantly faster execution

Task 2:

KV Caching Implementation and Benchmarking

KV Caching: Brief GPT-2 Example

In GPT-2, KV caching is implemented with:

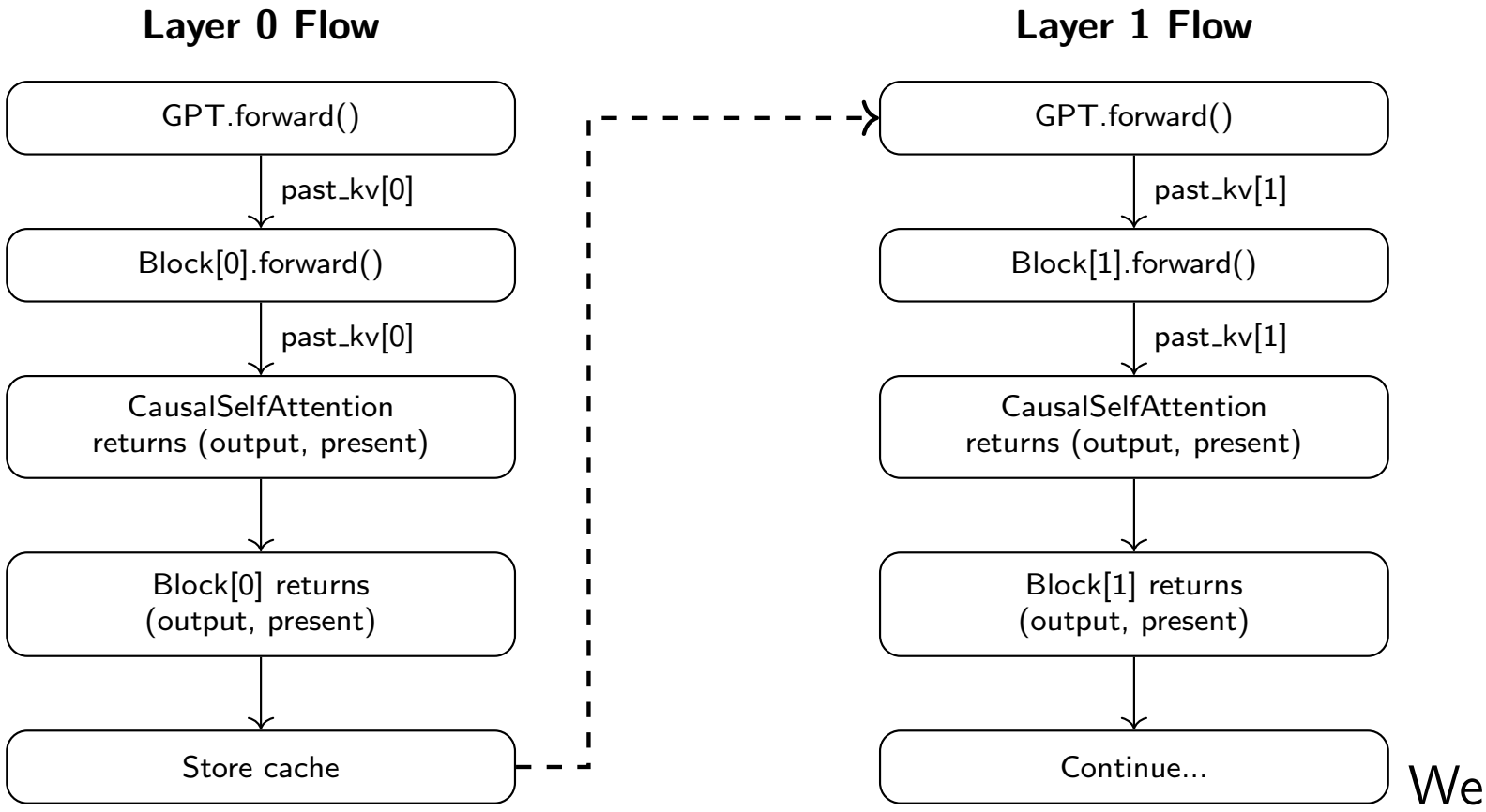
- Optional `past` argument containing cached K/V
- **Solution**: Cache previously computed key-value pairs

$$\text{past\_kv} = (\boldsymbol{k}_1, \ldots, \boldsymbol{k}_{T-1}, \boldsymbol{v}_1, \ldots, \boldsymbol{v}_{T-1})$$

- Shape: `[batch, 2, heads, sequence, features]`
- Cache concatenation logic:

```
if past is not None:
    pk, pv = tf.unstack(past, axis=1)
    k = tf.concat([pk, k], axis=-2)
    v = tf.concat([pv, v], axis=-2)
```

KV Cache Flow Across Layers



We need to modify `forward` logic in NanoGPT across three classes: `CausalSelfAttention`, `Block`, and `GPT`.

CausalSelfAttention.forward() changes

CausalSelfAttention: Change 1 - KV Cache Logic

Original NanoGPT

```
def forward(self, x):
    ...
    # reshape & transpose
    k = k.view(B,T,nh,hs)
        .transpose(1,2)
    q = q.view(B,T,nh,hs)
        .transpose(1,2)
    v = v.view(B,T,nh,hs)
        .transpose(1,2)

    # attention computation
    ...
    return y
```

Modified (KV caching)

```
def forward(self, x, past_kv=None):
    ...
    # reshape & transpose
    k = k.view(B,T,nh,hs)
        .transpose(1,2)
    q = q.view(B,T,nh,hs)
        .transpose(1,2)
    v = v.view(B,T,nh,hs)
        .transpose(1,2)
    # KV CACHE LOGIC
    if past_kv is not None:
        past_k, past_v = past_kv
        k = torch.cat([past_k,k],dim
            =2)
        v = torch.cat([past_v,v],dim
            =2)
    present = (k, v)
    # attention computation
    ...
    return y, present
```

CausalSelfAttention: Explanation of Change 1

Added `past_kv` argument:

- `None` for first token (no cache yet)
- Contains cached K/V for subsequent tokens

Cache concatenation logic (Exactly the same as GPT-2 logic):

- Extract past keys (`past_k`) and values (`past_v`)
- Concatenate with new K/V on sequence dimension (`dim=2`)
- Store updated cache as `present = (k, v)`

Shape changes with caching:

- K/V: $(B, nh, T, hs) \rightarrow (B, nh, T_{past} + T, hs)$
- Attention: $(B, nh, T, T) \rightarrow (B, nh, T, T_{past} + T)$
- Output: still (B, nh, T, hs) (only current tokens)

CausalSelfAttention: Change 2 - Causal Masking

Original NanoGPT

```
if self.flash:
    y = torch.nn.functional
        .scaled_dot_product_attention(
        q, k, v,
        is_causal=True)
else:
    att = (q @ k.transpose(-2,-1))
          * (1.0/math.sqrt(k.size(-1))
    att = att.masked_fill(
        self.bias[:,:,:T,:T]==0,
        float('-inf'))
...
return y
```

Modified (KV caching)

```
if self.flash:
    is_causal = past_kv is None
    y = torch.nn.functional
        .scaled_dot_product_attention(
        q, k, v,
        is_causal=is_causal)
else:
    att = (q @ k.transpose(-2,-1))
          * (1.0/math.sqrt(k.size(-1))
    if past_kv is not None:
        pass  # skip masking
    else:
        att = att.masked_fill(
            self.bias[:,:,:T,:T]==0,
            float('-inf'))
...
return y, present
```

CausalSelfAttention: Explanation of Change 2

Why modify causal masking?

Without KV cache (parallel processing):

```
Input: [A, B, C]  # All processed in parallel
A -> [A, B, C]    # A can see future tokens B, C (BAD!)
B -> [A, B, C]    # B can see future token C (BAD!)
C -> [A, B, C]    # Masking prevents information leakage
```

Causal masking is **required** to prevent future token visibility.

With KV cache (sequential processing):

```
Cache: [A, B, C]  # Past tokens already computed
Input: [D]        # Only new token
D -> [A, B, C, D] # D only sees past and itself (OK!)
```

Causal masking is **not needed** - single token attends to all past.

Implementation: Only apply masking when `past_kv is None`

Block.forward() changes

Block.forward: Modification

Original NanoGPT

```
def forward(self, x):
    x = x + self.attn(
            self.ln_1(x))
    x = x + self.mlp(
            self.ln_2(x))
    return x
```

Modified (KV caching)

```
def forward(self, x, past_kv=None):
    y, present = self.attn(
        self.ln_1(x),
        past_kv=past_kv)
    x = x + y
    x = x + self.mlp(
            self.ln_2(x))
    return x, present
```

Explanation:

- Added `past_kv` parameter to accept cached KV
- Unpack attention output: `self.attn` now returns 2 objects instead of one. `(y, present)`
- Split residual connection into two lines for clarity, logic remains
- Return both output and updated cache: `(x, present)`

GPT.forward() changes

GPT.forward: Change 1 - Absolute Position Tracking

Original NanoGPT

```
def forward(self, idx,
            targets=None):
    ...
    pos = torch.arange(
        0, t,
        dtype=torch.long,
        device=device)
    tok_emb = self.transformer
              .wte(idx)
    pos_emb = self.transformer
              .wpe(pos)
    x = self.transformer.drop(
        tok_emb + pos_emb)
    ...
```

Modified (KV caching)

```
def forward(self, idx,
    targets=None, past_kv=None):
    ...
    if past_kv is not None:
        past_length =
            past_kv[0][0].size(2)
        pos = torch.arange(
            past_length,
            past_length + t,
            dtype=torch.long,
            device=device)
    else:
        pos = torch.arange(
            0, t,
            dtype=torch.long,
            device=device)
    tok_emb = self.transformer
              .wte(idx)
    pos_emb = self.transformer
              .wpe(pos)
    x = self.transformer.drop(
        tok_emb + pos_emb)
    ...
```

GPT.forward: Explanation of Change 1

Problem: Position embeddings must match *absolute* positions

Example: Predicting "coolest" after ["I", "am", "the"]

Without KV cache:

```
idx = ["I", "am", "the", "coolest"]
pos = [0, 1, 2, 3]
pos_emb = wpe([0, 1, 2, 3])  # Correct absolute positions
```

With KV cache (naive):

```
idx = ["coolest"]  # Only new token
pos = [0]          # WRONG! Should be position 3
pos_emb = wpe([0]) # Uses embedding for position 0
```

Solution: Add `past_length` to position indices

GPT.forward: Change 2 - Cache Management

Original NanoGPT

```
for block in
    self.transformer.h:
    x = block(x)
x = self.transformer
    .ln_f(x)
```

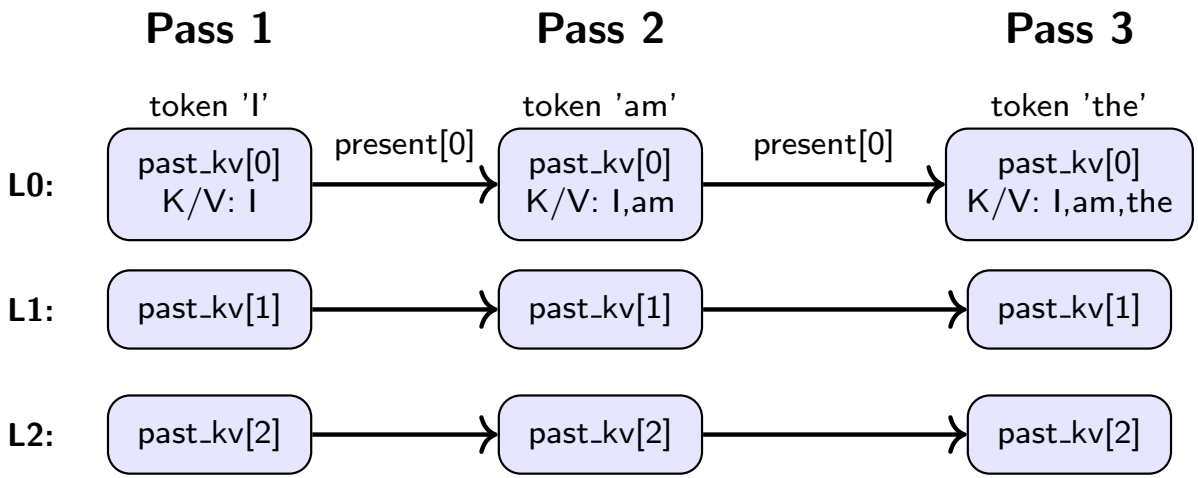
Modified (KV caching)

```
present_kvs = []
for i, block in enumerate(
    self.transformer.h):
    layer_past = past_kv[i] if
        past_kv is not None
        else None
    x, layer_present = block(
        x, past_kv=layer_past)
    present_kvs.append(
        layer_present)
x = self.transformer
    .ln_f(x)
```

Explanation:

- Initialise `present_kvs` list to store updated caches
- Fetch layer-specific cache: `layer_past = past_kv[i]`
- Block concatenates cached KV with new KV, returns updated cache
- Append new cache: `present_kvs.append(layer_present)`
- `present_kvs` becomes `past_kv` for next token

Cache Flow Visualisation Across Forward Passes



Each layer's cache flows **horizontally** (to itself in next pass), never mixing with other layers.

GPT.forward: Why Per-Layer Caching?

Each layer needs its own cache because:

- Each layer applies different learned transformations
- Produces unique K/V representations
- Cannot reuse another layer's cache

Cache flow:

1. Layer 0 processes input, returns cache[0]
2. Layer 1 processes Layer 0 output, returns cache[1]
3. Layer 2 processes Layer 1 output, returns cache[2]
4. ...continue for all layers
5. Next token: each layer reuses its own cache

Without per-layer caching, each layer would recompute attention over all previous tokens and make kv caching redundant.

Benchmarking Results

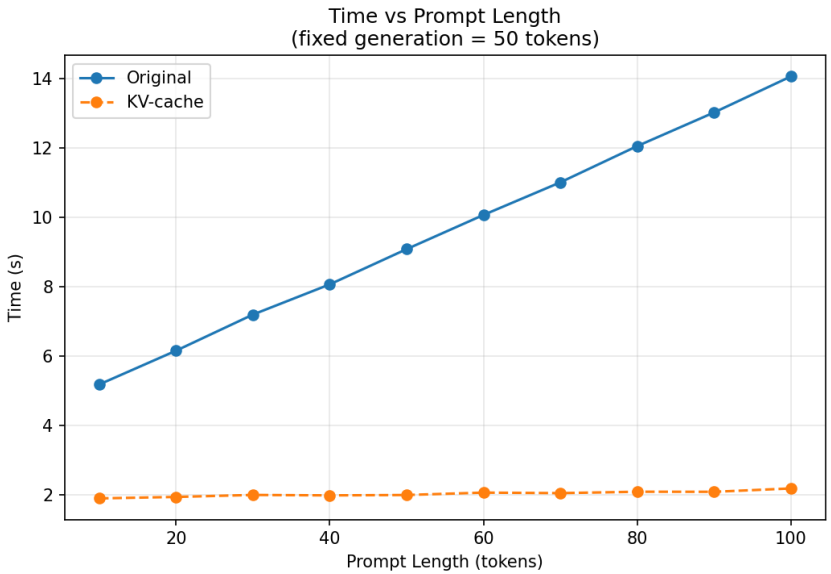
Benchmark Setup

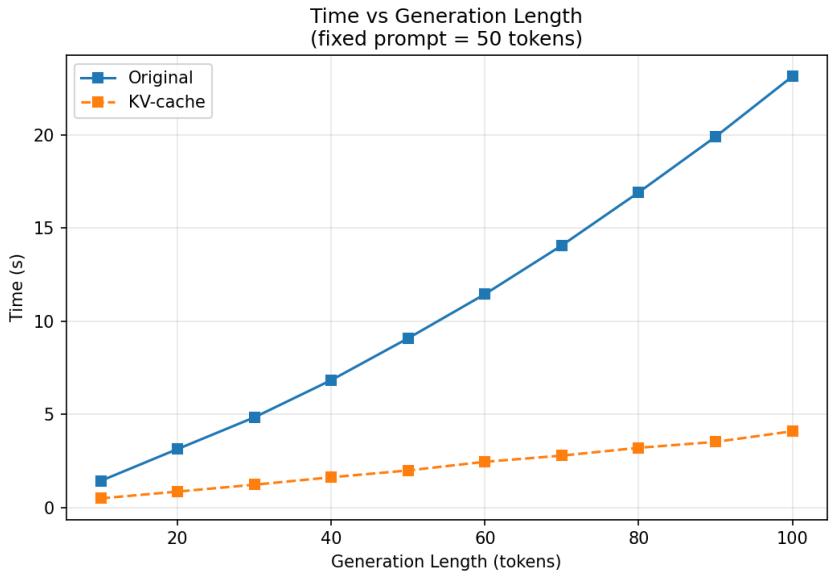
Test configurations:

- **Graph 1**: Vary prompt length (10-100 tokens, step = 10), fix generation = 50 tokens
- **Graph 2**: Vary generation length (10-100 tokens, step = 10), fix prompt = 50 tokens
- Model: GPT-2

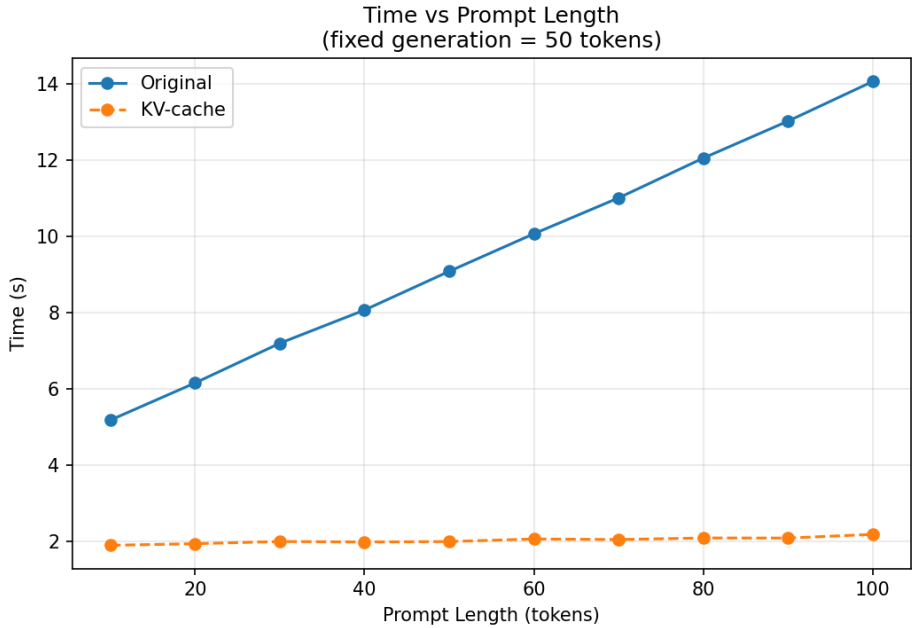
To evaluate how inference time scales with prompt length and generation length *before and after* KV caching.

Benchmark Results





Analysis: Time vs Prompt Length



Without KV caching:

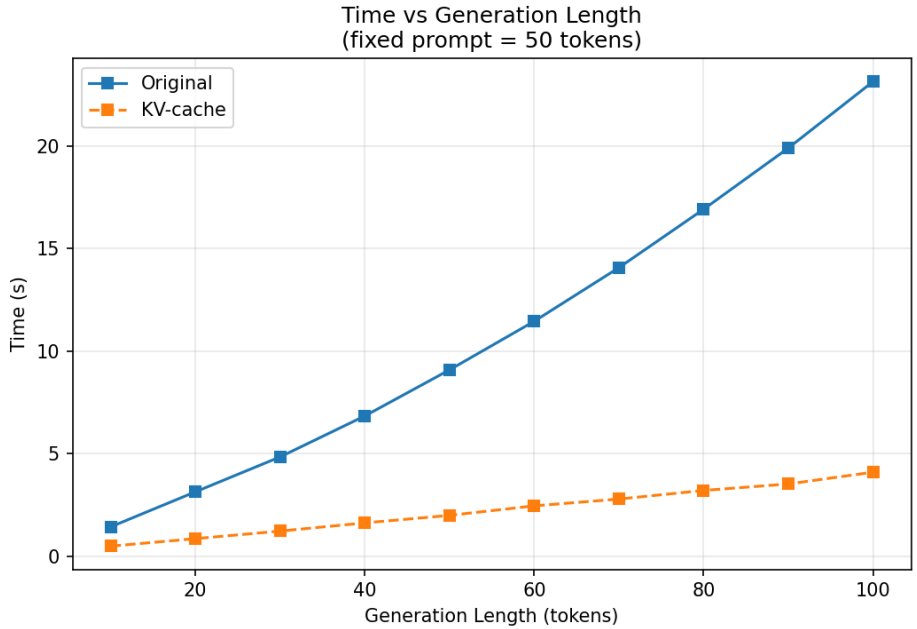
- Generation time increases **linearly** with prompt length
- Prompt is recomputed at every generation step (50 times!)

With KV caching:

- Generation time remains nearly **constant** ($\sim$2s)
- Prompt KV computed **once** and reused for all 50 steps

Key insight: KV caching makes generation time independent of prompt length

Analysis: Time vs Generation Length



Without KV caching:

- Generation time grows **quadratically** $O(n^2)$ with output length
- Each new token recomputes attention over all previous tokens
- Operations: $1 + 2 + 3 + \ldots + n = \frac{n(n+1)}{2}$

With KV caching:

- Generation time grows **linearly** $O(n)$
- Only new token's attention computed, cached KV reused

Summary

Multi-head Attention Implementation:

- NanoGPT uses combined projections and tensor reshaping for GPU efficiency
- Same mathematical operation as slides, but optimised for parallel computation

KV Caching Implementation:

- Modified three classes: `CausalSelfAttention`, `Block`, `GPT`
- Key changes: cache concatenation, conditional masking, absolute positioning
- Per-layer caching essential for correct computation

Performance Gains:

- Generation time nearly independent of prompt length
- Linear vs quadratic scaling with generation length
- Essential optimization for production inference

Key Takeaways

1. **Implementation matters**: Same math, different efficiency through code optimization
2. **KV caching is essential**: Drastically reduces time complexity
3. **Cache management is difficult man**: Requires careful indexing and per-layer tracking
4. **Real-world impact**: Makes long-context generation practical

Thank you!